

TỔNG HỢP CÁC CA KIỂM THỬ HIỆU NĂNG & CHỨC NĂNG (TEST CASES SPECIFICATION)

Mã bài tập: HW05 – Performance Testing (Exercise ID: HW05-AI)

Sinh viên thực hiện: BaoBeiii – MSSV: 23127327

Hệ thống kiểm thử (SUT): EShop System (eshop-sut)

Công cụ: k6 v2.1.0 & Apache JMeter 5.6.3

Ngày cập nhật: 2026-09-02

1. Ma Trận Ảnh Xạ Kiểm Thử (Traceability Matrix)

Mã Ca Kiểm Thử	Phân Hệ / Luồng Nghiệp Vụ	Endpoint Mục Tiêu	Phương Thức	Tham Số Hóa (Data Source)	Kịch Bản Áp Dụng
TC-AUTH-01	Đăng nhập thành công (Happy Path)	/api/login	POST	data/users.csv (user1..user50)	Load, Stress, Spike, Endurance
TC-AUTH-02	Đăng nhập sai mật khẩu	/api/login	POST	data/users.csv (invalid_pass@eshop.com)	Stress
TC-AUTH-03	Khóa tài khoản Lockout (FR-02)	/api/login	POST	data/users.csv (wrong_user@eshop.com)	Stress (Bắt lỗi BUG-04)
TC-PROD-01	Duyệt toàn bộ danh mục sản phẩm	/api/products	GET	Không	Load, Stress, Spike, Endurance
TC-PROD-02	Xem chi tiết sản phẩm ngẫu nhiên	/api/products/:id	GET	data/products.csv (id: 1..15)	Load, Stress, Spike, Endurance (Bắt BUG-03)
TC-PROD-03	Tìm kiếm sản phẩm theo từ khóa	/api/products?search=	GET	data/products.csv (keyword)	Bắt lỗi hỏng BUG-02
TC-CPN-01	Áp dụng coupon phần trăm (SAVE10)	/api/apply-coupon	POST	data/orders.csv (SAVE10)	Load, Stress, Spike (Bắt BUG-01)
TC-CPN-02	Áp dụng coupon tiền mặt (BIGBUY)	/api/apply-coupon	POST	data/orders.csv (BIGBUY)	Load, Stress, Spike
TC-CPN-03	Áp dụng coupon VIP đơn giá cao (VIP100)	/api/apply-coupon	POST	data/orders.csv (VIP100)	Load, Stress, Spike
TC-CPN-04	Kiểm tra coupon không đủ điều kiện	/api/apply-coupon	POST	Đơn hàng \$ < 500.000\$ đ + VIP100	Load (Kiểm tra HTTP 400)
TC-ORD-01	Tạo đơn hàng mới (Checkout)	/api/checkout	POST	data/orders.csv (shipping_address)	Load, Stress, Spike, Endurance

Mã Ca Kiểm Thử	Phân Hệ / Luồng Nghiệp Vụ	Endpoint Mục Tiêu	Phương Thức	Tham Số Hóa (Data Source)	Kịch Bản Áp Dụng
TC-ORD-02	Ghi nhận sử dụng mã giảm giá	/api/coupon-usage	POST	coupon_id lấy từ step coupon	Load, Stress, Spike, Endurance
TC-ORD-03	Truy vấn lịch sử đơn hàng cá nhân	/api/orders/my-orders	GET	JWT Token của người dùng	Load, Stress, Spike, Endurance
TC-PERF-01	Kiểm thử Tải cao điểm (Load Test)	Toàn bộ luồng E2E	k6 / JMX	Đầy đủ 3 file CSV	Peak 50 VUs (8 phút)
TC-PERF-02	Kiểm thử Điểm gãy (Stress Test)	Toàn bộ luồng E2E	k6 / JMX	Đầy đủ 3 file CSV	Bậc thang 10..200 VUs
TC-PERF-03	Kiểm thử Đột biến tải (Spike Test)	Toàn bộ luồng E2E	k6 / JMX	Đầy đủ 3 file CSV	Đột biến 150 VUs trong 10s
TC-PERF-04	Kiểm thử Độ bền (Endurance Test)	Toàn bộ luồng E2E	k6	Đầy đủ 3 file CSV	Bền vững 30 VUs (15 phút)

2. Chi Tiết Các Ca Kiểm Thử Chức Năng & Điểm Biên (Functional & Edge Cases)

Nhóm 1: Xác Thực & Quản Lý Phiên (Auth-Heavy)

- ◆ TC-AUTH-01: Đăng nhập tài khoản hợp lệ (Happy Path)
 - **Mục tiêu:** Xác thực người dùng hợp lệ, cấp phát JWT token để thực hiện các bước sau.
 - **Đầu vào (Input):** email , password từ file data/users.csv .
 - **Kỳ vọng (Expected Result):**
 - HTTP Status: 200 OK .
 - Body JSON chứa thuộc tính token dạng JWT string và object user có id , email , role .
 - Thời gian phản hồi (\$p95\$): \$ < 1500\text{ms}\$.
- ◆ TC-AUTH-02: Đăng nhập sai mật khẩu (Invalid Credentials)
 - **Mục tiêu:** Kiểm tra phản hồi lỗi khi cung cấp sai thông tin xác thực.

- **Đầu vào:** `email: "invalid_pass@eshop.com"` , `password: "BadPassword999!"` .
- **Kỳ vọng:**
 - HTTP Status: `401 Unauthorized` .
 - Body JSON: `{"error": "Invalid email or password"}` .
 - Trường `login_attempts` trong CSDL của user tăng thêm.

◆ TC-AUTH-03: Kích hoạt cơ chế khóa tài khoản FR-02 (Lockout Triggering)

- **Mục tiêu:** Kiểm tra cơ chế tự động tạm khóa tài khoản sau các lần đăng nhập thất bại liên tiếp.
- **Đầu vào:** Gửi request đăng nhập sai mật khẩu 3 lần liên tiếp cho `wrong_user@eshop.com` .
- **Kỳ vọng theo đặc tả FR-02:**
 - Lần sai 1 & 2: HTTP `401` .
 - Lần sai thứ 3: Cột `locked_until` được gán mốc thời gian `+$30$` giây, các lần thử tiếp theo trong 30s trả về HTTP `403 Forbidden` (`{"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."}`).
- **Thực tế phát hiện (BUG-04):**
 - Code SUT tăng `+2` thay vì `+1` , khiến ngay lần sai thứ 2 tài khoản đã bị khóa, và thời gian khóa bị gán thành 180.000ms (3 phút) thay vì 30 giây.
 - Bẫy k6 check: `r.status === 403` được kích hoạt và ghi nhận lỗi.

Nhóm 2: Đọc Danh Mục & Chi Tiết Sản Phẩm (Read-Heavy)

◆ TC-PROD-01: Duyệt danh sách toàn bộ sản phẩm

- **Mục tiêu:** Đọc nhanh toàn bộ danh mục hàng hóa trên hệ thống.
- **Đầu vào:** Không có query parameter (`GET /api/products`).
- **Kỳ vọng:**
 - HTTP Status: `200 OK` .
 - Body trả về JSON Array chứa danh sách các sản phẩm (tối thiểu 10 sản phẩm mẫu).
 - Thời gian phản hồi (`$p95$`): `$ < 800\text{ms}$`.

◆ TC-PROD-02: Xem chi tiết sản phẩm & Kiểm tra kiểu dữ liệu

- **Mục tiêu:** Truy xuất thông tin chi tiết một sản phẩm theo `id` .
- **Đầu vào:** `id` từ `1` đến `15` lấy từ `data/products.csv` .
- **Kỳ vọng theo tiêu chuẩn:**
 - HTTP Status: `200 OK` .

- Thuộc tính `price` phải luôn là kiểu số (`typeof price === 'number'`).
 - **Thực tế phát hiện (BUG-03):**
 - Với các sản phẩm có ID chẵn (2, 4, 6...), `price` bị ép kiểu thành `string` (`"350000"`).
 - Kịch bản k6 đặt assertion: `typeof r.json('price') === 'number'` để bẫy và ghi nhận tỷ lệ vi phạm.
 - ◆ **TC-PROD-03: Tìm kiếm sản phẩm theo từ khóa (Security & Edge Case)**
 - **Mục tiêu:** Đánh giá tính an toàn khi tìm kiếm và xử lý đầu vào người dùng.
 - **Đầu vào:** `GET /api/products?search=áo` và test ký tự đặc biệt `áo' OR 1=1--`.
 - **Kỳ vọng:** Hệ thống thực thi Parameterized query an toàn, trả về kết quả mảng JSON chứa các sản phẩm phù hợp.
 - **Thực tế phát hiện (BUG-02):** Nối chuỗi trực tiếp SQL gây lỗi SQLite vắng trang HTML HTTP 500.
-

Nhóm 3: Kiểm Tra & Áp Dụng Mã Giảm Giá (Coupon Validation)

- ◆ **TC-CPN-01: Áp dụng coupon phần trăm (`SAVE10`)**
 - **Mục tiêu:** Giảm 10% cho mọi giá trị đơn hàng hợp lệ.
 - **Đầu vào:** `code: "SAVE10"`, `total_amount: 200000`, `user_id: 1`.
 - **Kỳ vọng theo nghiệp vụ:**
 - `discount_amount` \$= 20.000\$ đ (\$200.000 \times 10\%).
 - `final_amount` \$= 180.000\$ đ (\$200.000 - 20.000\$).
 - **Thực tế phát hiện (BUG-01):**
 - Công thức sai `total_amount * (1 - discount_value)` tính ra discount âm `$-1.800.000$ đ` làm `final_amount` vọt lên `$2.000.000$ đ`.
 - Bẫy k6 check: `r.json('final_amount') < totalAmount && r.json('discount_amount') > 0` bắt trúng 100% bug này.
- ◆ **TC-CPN-02: Áp dụng coupon cố định có điều kiện tối thiểu (`BIGBUY`)**
 - **Mục tiêu:** Giảm cố định 50.000đ cho đơn hàng từ 300.000đ trở lên.
 - **Đầu vào:** `code: "BIGBUY"`, `total_amount: 350000`.
 - **Kỳ vọng:**
 - HTTP `200 OK`.
 - `discount_amount = 50000`, `final_amount = 300000`.

◆ TC-CPN-03: Áp dụng coupon VIP giá trị lớn (VIP100)

- **Mục tiêu:** Giảm 100.000đ cho đơn hàng từ 500.000đ trở lên.
 - **Đầu vào TH1 (Hợp lệ):** `code: "VIP100", total_amount: 650000` $\rightarrow$ Giảm 100.000đ, trả \$550.000\$ đ.
 - **Đầu vào TH2 (Không đủ điều kiện):** `code: "VIP100", total_amount: 150000` $\rightarrow$ HTTP 400 Bad Request ({"error": "Đơn hàng chưa đủ giá trị tối thiểu 500,000 đ để áp dụng mã này"}).
-

Nhóm 4: Giao Dịch Đặt Hàng & Ghi Nhận (Transactional)

◆ TC-ORD-01: Thanh toán và tạo đơn hàng (POST /api/checkout)

- **Mục tiêu:** Tạo đơn hàng mới trong cơ sở dữ liệu.
- **Đầu vào:** Header `Authorization: Bearer <token>`, Body: `{"total_amount": 150000, "shipping_address": "123 Test St, Q1, HCMC"}`.
- **Kỳ vọng:**
 - HTTP Status: 200 OK.
 - Body JSON: `{"message": "Checkout successful", "orderId": <integer>}`.
 - Thời gian phản hồi (\$p95\$): $\$ < 1500\text{ms}$.

◆ TC-ORD-02: Ghi nhận lịch sử sử dụng coupon (POST /api/coupon-usage)

- **Mục tiêu:** Lưu thông tin người dùng đã sử dụng mã vào bảng `coupon_usage` để khống chế giới hạn số lần sử dụng.
- **Đầu vào:** Header chứa JWT Token, Body: `{"coupon_id": 1}`.
- **Kỳ vọng:** HTTP Status: 200 OK, Body: `{"message": "Usage recorded"}`.

◆ TC-ORD-03: Truy vấn danh sách đơn hàng cá nhân (GET /api/orders/my-orders)

- **Mục tiêu:** Kiểm tra dữ liệu đơn hàng vừa tạo đã xuất hiện trong danh sách cá nhân.
 - **Đầu vào:** Header chứa JWT Token của người dùng.
 - **Kỳ vọng:** HTTP Status: 200 OK, trả về mảng danh sách các đơn hàng đã đặt của đúng `user_id`.
-

3. Chi Tiết Các Ca Kiểm Thử Hiệu Năng Hệ Thống (Performance Test Scenarios)

◆ TC-PERF-LOAD: Peak Traffic Load Testing (50 VUs)

- **Kịch bản:** Mô phỏng lưu lượng người dùng truy cập đồng thời trong giờ cao điểm bình thường của EShop.
- **Cấu hình Stages:**
 - 0m -> 2m : Tăng dần từ 0 lên 50 VUs (Ramp-up).
 - 2m -> 7m : Duy trì liên tục tại 50 VUs trong 5 phút (Steady state).
 - 7m -> 8m : Giảm dần từ 50 về 0 VUs (Ramp-down).
- **Tiêu chí Đạt (SLAs / Pass Criteria):**
 - Tỷ lệ lỗi (\$Error\text{Rate}\$): $< 5\%$.
 - Trễ đọc (\$p95\$ Read: `/api/products`): $< 800\text{ms}$.
 - Trễ ghi (\$p95\$ Write: `/api/login`, `/api/checkout`): $< 1500\text{ms}$.
 - Bộ nhớ RAM tiến trình `node.exe`: Tăng ổn định và giữ nguyên trần, không bị rò rỉ.

◆ TC-PERF-STRESS: Stepped Ramp-up Stress Testing (200 VUs)

- **Kịch bản:** Đẩy áp lực tải vượt ngưỡng thiết kế để tìm điểm gãy (Breaking Point) của backend Node.js và SQLite.
- **Cấu hình Bậc thang (Stepped Stages):**
 - Bậc 1: 10 VUs trong 45s.
 - Bậc 2: 30 VUs trong 45s.
 - Bậc 3: 70 VUs trong 45s.
 - Bậc 4: 120 VUs trong 45s.
 - Bậc 5: 200 VUs trong 45s (Ngưỡng bão hòa tối đa).
 - Hạ tải: Về 0 VUs trong 30s.
- **Mục tiêu Quan sát:**
 - Xác định mức VU mà tại đó Error Rate vượt quá 5% hoặc CPU máy đạt 100% .
 - Ghi nhận lỗi tranh chấp khóa cơ sở dữ liệu SQLite (`SQLITE_BUSY` hoặc HTTP 500).

◆ TC-PERF-SPIKE: Sudden Surge Flash Sale (150 VUs)

- **Kịch bản:** Đột biến lượng truy cập cực nhanh trong thời gian rất ngắn (mô phỏng mở bán Flash Sale 0h).
- **Cấu hình:**
 - 0s -> 30s : Tải nền thấp 5 VUs.

- 30s -> 40s : Đột ngột tăng vọt lên 150 VUs trong đúng 10 giây!
- 40s -> 70s : Giữ đỉnh 150 VUs trong 30 giây.
- 70s -> 80s : Hạ đột ngột về 5 VUs trong 10 giây.
- 80s -> 110s : Quan sát thời gian phục hồi (Recovery Time) trong 30 giây.
- **Tiêu chí Đánh giá:**
 - Hệ thống có bị sập hoàn toàn (Crash tiến trình `node.exe`) hay không?
 - Sau khi hạ tải về 5 VUs, thời gian phản hồi có quay về mức bình thường ($< 800\text{ms}$) trong vòng bao nhiêu giây?

◆ TC-PERF-ENDUR: Endurance / Soak Testing (15 Phút)

- **Kịch bản:** Kiểm tra tính ổn định lâu dài dưới tải vừa phải để phát hiện rò rỉ tài nguyên (Memory Leak, Connection Leak).
- **Cấu hình:** Duy trì liên tục 30 VUs trong 15 phút.
- **Tiêu chí Đánh giá:**
 - So sánh dung lượng RAM (Private Bytes / Heap Used) ở phút thứ 2 và phút thứ 14: Tỷ lệ tăng trưởng trôi (Drift) không được vượt quá 10% .
 - Đường biểu diễn Response Time phải giữ nguyên phương ngang, không có xu hướng dốc lên theo thời gian.